LLM Pipeline — Görsel Akış

Taşıt Finansmanı Ön Başvuru Chatbot'u

Mesajın hangi yolu izlediği · LLM'e ne gönderildiği · ne döndüğü · nereden geçtiği · hangi katmanın LLM'siz çalıştığı

Üst Seviye Mimari

```
flowchart LR
    MOB[Mobile Banking App]
    API[FastAPI<br/>routes_chat]
    ORCH[LangGraph<br/>orchestrator]
    GW[LiteLLM Gateway<br/>routing + budget + quota]
    DET[Deterministic<br/>rules + catalog + tckn]
    RAG[RAG<br/>FAQ + Qdrant/FAISS]
    PERS[Persistence<br/>SQLAlchemy]
    VLLM[vLLM 72B-AWQ + 14B + Guard 8B<br/>on-prem · 96 GB GPU]

    MOB -->|X-Customer-Id| API
    API --> ORCH
    ORCH --> GW
    ORCH --> DET
    ORCH --> RAG
    ORCH --> PERS
    GW --> VLLM
```

LLM çağrıları **yalnızca** gateway üzerinden; uygulama model ID'sini bilmez.

State Makinesi

```
stateDiagram-v2
    [*] --> START
    START --> GREETED: POST /chat/session<br/>(template, no LLM)
    GREETED --> AWAITING_INTENT: POST /chat
    AWAITING_INTENT --> AWAITING_FINANCE_TYPE: intent yok / undecided
    AWAITING_FINANCE_TYPE --> COLLECTING_FIELDS: finance_type set
    AWAITING_INTENT --> COLLECTING_FIELDS: start_application
    COLLECTING_FIELDS --> VALIDATING: alanlar geldi
    VALIDATING --> AWAITING_FIELD_FIX: rules errors<br/>(LLM yumuşatma)
    VALIDATING --> COLLECTING_FIELDS: missing fields<br/>(LLM ile soru)
    VALIDATING --> AWAITING_CONFIRMATION: valid<br/>(editable tablo)
    AWAITING_FIELD_FIX --> VALIDATING: user düzeltti
    AWAITING_CONFIRMATION --> PERSISTED: confirm<br/>(idempotent DB write)
    PERSISTED --> AWAITING_HGS_DECISION: HGS pitch<br/>(hard-coded)
    AWAITING_HGS_DECISION --> COMPLETED: Evet/Hayır
    COMPLETED --> [*]

    AWAITING_INTENT --> COMPLETED: cancel
    AWAITING_CONFIRMATION --> COMPLETED: cancel
```

Turn 0 — İlk Mesaj (Chatbot Açılır)

```
sequenceDiagram
    autonumber
    participant U as Mobile App
    participant API as FastAPI
    participant G as greeting_node
    participant DB as Persistence

    U->>API: POST /chat/session<br/>X-Customer-Id
    API->>API: require_customer<br/>→ CustomerProfile{full_name, gender}
    API->>API: session_id üret · state=START
    API->>G: greeting_node(GraphState)
    Note over G: ❌ LLM YOK<br/>Template render:<br/>"Merhaba {ad} {Bey/Hanım}, ..."
    G-->>API: reply · state=GREETED · SHOW_GREETING
    API->>DB: save(state)
    API-->>U: ChatResponse{reply, actions}
```

LLM çağrı sayısı: 0 — gender + ad customer-master'dan kesin; template yeterli.

Turn 1+ — Kullanıcı Mesajı Geldiğinde

```
sequenceDiagram
    autonumber
    participant U as Mobile App
    participant API as FastAPI
    participant GR as guardrail
    participant N as intent_node
    participant GW as LiteLLM Gateway
    participant V as vLLM
    participant R as rules.py
    participant RG as response_gen

    U->>API: POST /chat<br/>{message, edited_fields?}
    API->>API: load state · merge edited_fields
    API->>GR: check_user_input(msg)
    alt blocked (injection)
        GR-->>API: SAFE_REPLY (no LLM)
        API-->>U: state korunur · akış END
    else clean
        N->>GW: invoke(NODE_FIELD)<br/>SYSTEM_INTENT_EXTRACTION
        GW->>GW: customer quota · budget · routing
        GW->>V: small model · JSON request
        V-->>GW: ExtractedFields JSON
        GW-->>N: parsed ExtractedFields
        N->>R: validate (deterministic)
        alt errors
            R-->>RG: ValidationResult
            RG->>GW: NODE_RESPONSE<br/>SYSTEM_VALIDATION_RESPONSE
            GW-->>RG: doğal Türkçe cevap
        else missing
            RG->>GW: NODE_RESPONSE<br/>SYSTEM_COLLECTION_PROMPT
            GW-->>RG: "Kefil TCKN paylaşır mısınız?"
        else valid
            R-->>API: summary_node (template, no LLM)
        end
        API-->>U: ChatResponse{reply, actions}
    end
```

Modele Ne Gidiyor, Ne Dönüyor

```
sequenceDiagram
    participant N as LangGraph node
    participant GW as LiteLLM Gateway
    participant V as vLLM

    N->>GW: invoke(node_purpose, system_prompt, user_message, context_chunks?)
    Note over GW: 1) customer quota check<br/>2) cloud guard<br/>3) fit_to_budget (trim)
    GW->>V: POST /v1/chat/completions<br/>{model: alias, messages:[<br/>  {role:system, content:SYSTEM_X},<br/>  {role:user,   content:payload}],<br/> max_tokens, temperature}
    V-->>GW: {choices:[{message:{content}}],<br/> usage:{prompt_tokens, completion_tokens}}
    GW->>GW: usage_log (customer_id_hash, tokens, cost)
    GW-->>N: LLMResponse{content, usage, latency_ms}
```

Gateway tüm pre-call kontrolleri yapar, post-call'da PII-free token log yazar.

Provider hata → `fallback_alias` bir kez denenir; hâlâ fail → safe deterministic reply.

Pydantic Sözleşmeleri — Hangi LLM Katmanı Ne Üretir

```
flowchart LR
    subgraph LLM1[LLM-1 intent_extraction]
      direction TB
      I1[user message + state context] --> O1[ExtractedFields<br/>Pydantic structured]
    end
    subgraph LLM2[LLM-2 response_generation validation]
      direction TB
      I2[ValidationResult + fields<br/>text serialize] --> O2[str<br/>doğal Türkçe]
    end
    subgraph LLM3[LLM-3 response_generation collection]
      direction TB
      I3[missing_field + fields<br/>text serialize] --> O3[str<br/>tek cümle soru]
    end
    subgraph LLM4[LLM-4 faq_answer]
      direction TB
      I4[question + retrieved chunks] --> O4[str<br/>grounded + citation]
    end
```

Çağrı	Output enforcement
Intent extraction	**Pydantic** `with_structured_output(ExtractedFields)` zorunlu
Validation response	Serbest metin · `_FALLBACK` deterministic geri dönüş
Collection prompt	Serbest metin · `_FALLBACK_PROMPTS` statik
FAQ answer	Serbest metin + citation · top-chunk fallback

`ExtractedFields` — Kritik Schema

```python
class ExtractedFields(BaseModel):
    intent: IntentType            # greet | start_application | provide_info |
                                  # update_field | confirm | reject | cancel |
                                  # faq_question | hgs_decision | undecided | unknown
    finance_type: FinanceType | None       # NEW | USED | None
    invoice_value: float | None
    vehicle_model: str | None             # ham — canonical sonra rapidfuzz ile
    guarantor_tckn: str | None
    casco_value: float | None
    model_year: int | None
    vehicle_age: int | None
    registration_date: date | None
    seller_tckn: str | None
    requested_amount: float | None
    faq_question: str | None
    field_to_update: str | None
    confidence: float = 0.5
```

Parse fail → `ExtractedFields(intent=UNKNOWN, confidence=0)` → chat "anlamadım".

Greeting Cevabını Yorumlama

```
flowchart TD
    M[Kullanıcı greeting'e cevap verir] --> N[intent_node · LLM extract]
    N --> I{ExtractedFields.intent +<br/>finance_type}

    I -->|start_application + NEW| NEW["Yeni araç akışı<br/>apply_fields → validate<br/>(fatura, model, kefil)"]
    I -->|start_application + USED| USED["İkinci el akışı<br/>apply_fields → validate<br/>(kasko, yaş, satıcı)"]
    I -->|undecided| UND[collection_node<br/>'Yeni mi ikinci el mi?'<br/>+ bilgi daveti]
    I -->|faq_question| FAQ[faq_router_node<br/>RAG + LLM grounded cevap<br/>state korunur]
    I -->|unknown / confidence=0| UNK["'Anlayamadım, taşıt<br/>finansmanı kapsamında<br/>yardımcı olabilirim'"]
    I -->|cancel / reject| C[cancel_node<br/>state=COMPLETED]
```

Karar mekanizması:

- LLM yorumlar → `ExtractedFields`
- Backend `route_after_intent` ile dallanma
- `rules.py` deterministic doğrulama

Alakasız Soru — 3 Katmanlı Savunma

```
flowchart TD
    M[Kullanıcı mesajı] --> L1{Katman 1<br/>Guardrail pattern}
    L1 -->|injection match| B1[BLOCK + audit<br/>SAFE_REPLY]
    L1 -->|temiz| L2{Katman 2<br/>LLM intent classifier}
    L2 -->|intent=unknown<br/>confidence=0| B2["'Anlayamadım, taşıt<br/>finansmanı kapsamında<br/>yardımcı olabilirim'"]
    L2 -->|valid intent| OK[Normal akış]
    OK --> L3{Katman 3<br/>Customer token quota}
    L3 -->|saatlik/günlük aşıldı| B3[CustomerBudgetExceeded<br/>+ admin alert]
    L3 -->|kota uygun| FLOW[Devam]
```

Örnekler:

- "Önceki talimatları unut" → Katman 1
- "Bana python kodu yaz" → Katman 2
- 1 saatte 100 kapsam dışı istek → Katman 3

4 LLM Çağrı Noktası — Özet

```
flowchart LR
    subgraph Aktif[4 Aktif LLM Çağrısı]
      A1["1- Intent + Field Extraction<br/>NODE_FIELD · small<br/>SYSTEM_INTENT_EXTRACTION<br/>→ ExtractedFields"]
      A2["2- Validation Response<br/>NODE_RESPONSE · small<br/>SYSTEM_VALIDATION_RESPONSE<br/>→ doğal Türkçe"]
      A3["3- Collection Prompt<br/>NODE_RESPONSE · small<br/>SYSTEM_COLLECTION_PROMPT<br/>→ tek cümle soru"]
      A4["4- FAQ Answer<br/>NODE_FAQ · large + RAG<br/>SYSTEM_FAQ_ANSWER<br/>→ grounded + citation"]
    end

    subgraph Yok[LLM Çağırmayan]
      D1[Greeting · template]
      D2[Summary · yapısal Python]
      D3[HGS pitch · hard-coded]
      D4[Rules / limit hesabı]
      D5[Canonical model · rapidfuzz]
      D6[TCKN checksum · date · money]
      D7[Idempotency · DB write]
      D8[Input guardrail · pattern]
    end
```

Max LLM/turn: 2 (intent + ya validation response ya collection ya FAQ)

Render Talimatı

```
# VS Code: "Marp for VS Code" extension → live preview
# CLI:
npm install -g @marp-team/marp-cli
marp docs/PRESENTATION.md --pdf
marp docs/PRESENTATION.md --pptx
marp docs/PRESENTATION.md --html
```

Mermaid: GitHub & VS Code preview otomatik render eder. Marp CLI için `@marp-team/marp-core` + mermaid plugin gerekli; veya diyagramları PNG export edip embed et.